Homework 3 Written Questions

CSCI 1430

Template Instructions

This document is a template with specific answer regions and a fixed number of pages. Given large class sizes and limited TA time, the template helps the course staff to grade efficiently and still focus on the content of your submissions. Please help us in this task:

- Make this document anonymous.
- Questions are in the orange boxes. Provide answers in the green boxes.
- Use the footer to check for correct page alignment.
- **Do NOT remove the answer box.**
- **Do NOT change the size of the answer box.**
- **Extra pages are not permitted unless otherwise specified.**
- **Template edits or page misalignment will lead to a 10 point deduction.**

Gradescope Submission

- Compile this document to a PDF and submit it to Gradescope.
- Pages will be automatically assigned to the right questions on Gradescope.

This Homework

- 3 questions **[8 + 13 + 14 = 35 points]**.
- Include code, images, and equations where appropriate.

Q1 — [8 points]

In the pinhole camera model, we represent a camera by a 3×4 camera matrix $\mathbf{M}$ that maps a 3D world point $\mathbf{P} = [X, Y, Z]^\top$ to a 2D image point $\mathbf{p} = [u, v]^\top$ via homogeneous coordinates:

$$
\begin{aligned}
w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} &= \mathbf{M} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \\
&= \underbrace{\begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{K}\text{ (intrinsic)}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}}_{[\mathbf{R}\,|\,\mathbf{t}]\text{ (extrinsic)}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}.
\end{aligned}
\tag{1}
$$

w is a scale factor that represents projective ambiguity, and is eliminated by dividing through (perspective division) to produce the final pixel space coordinates. $\mathbf{K}$ encodes the intrinsic parameters (sensor-ratio-adjusted focal lengths f_x, f_y, principal point c_x, c_y, skew s). Extrinsic matrix $[\mathbf{R}\,|\,\mathbf{t}]$ transforms 3D points from world coordinates into camera coordinates.

Q1.1 — [5 points]

Let's look at the pinhole camera demo from class. Play with the demo!

```
$ cd questions/camerademo
$ uv run python liveCamera.py
```

For each change to the camera parameters listed in the rows below, place ■ in the columns whose properties hold. There may be multiple for each row, or none.

- Assume a 3D scene with points at varying depths.
- "u, v changes" means that coordinate of the projected point is affected.
- "Same pixel amount" means every projected point shifts that number of pixels.
- "Induces parallax" means a nearby and a distant point that overlap before the change will separate afterward.

		u **changes**	v **changes**	**Same pixel amount**	**Induces parallax**
Intrinsic changes					
Double f_x, f_y	$f_x* = 2,\ f_y* = 2$	☐	☐	☐	☐
Double f_x only	$f_x* = 2$	☐	☐	☐	☐
Double f_y only	$f_y* = 2$	☐	☐	☐	☐
Increase skew	$s+ = \delta$	☐	☐	☐	☐
Shift principal point	$c_x+ = 50$	☐	☐	☐	☐
Extrinsic changes					
Translate camera right	t_x increases	☐	☐	☐	☐
Translate camera up	t_y increases	☐	☐	☐	☐
Translate camera forward	t_z increases	☐	☐	☐	☐
Rotate around x-axis (tilt)	$\theta_x = 25°$	☐	☐	☐	☐
Rotate around y-axis (pan)	$\theta_y = 25°$	☐	☐	☐	☐
Rotate around z-axis (roll)	$\theta_z = 25°$	☐	☐	☐	☐

Q1.2 — [2 points]

Consider the *dolly zoom* or Vertigo effect (Hitchcock, 1958; a thriller; the two 'staircase' scenes). It maintains a subject on one depth plane while subjects on other depth planes grow or shrink. Here's another example from Goodfellas (Scorsese, 1990; a gangster movie).
Using the $\mathbf{K}, \mathbf{R}, \mathbf{t}$ parameters and the properties from our table, explain how this effect is achieved and why it is possible. **[3–5 sentences]**

TODO: Your answer here.

Q1.3 — [1 point]

Where is the camera center $\mathbf{C}$ in world coordinates? Use ■.
Hint 1: The extrinsic acts as $\mathbf{R}\,\mathbf{C} + \mathbf{t} = \mathbf{0}$, since $\mathbf{C}$ is the world point that projects to the origin in camera coordinates.
Hint 2: $\mathbf{R}^\top = \mathbf{R}^{-1}$.

(a) ☐ $\mathbf{C} = \mathbf{t}$
(b) ☐ $\mathbf{C} = -\mathbf{t}$
(c) ☐ $\mathbf{C} = \mathbf{R}\,\mathbf{t}$
(d) ☐ $\mathbf{C} = -\mathbf{R}\,\mathbf{t}$
(e) ☐ $\mathbf{C} = \mathbf{R}^\top\mathbf{t}$
(f) ☐ $\mathbf{C} = -\mathbf{R}^\top\mathbf{t}$

Q2 — [13 points]
Let's calibrate our camera. The class demo for this question is *liveCalibration.py*.

We want to estimate $\mathbf{M}$ from N known 2D–3D point correspondences $\{(\mathbf{p}_i, \mathbf{P}_i)\}$. Writing $\mathbf{m}_1^\top, \mathbf{m}_2^\top, \mathbf{m}_3^\top$ for the three rows of $\mathbf{M}$ (each a 1×4 vector), this gives three scalar equations:

$$\mathbf{m}_1^\top \mathbf{P}_i^h = w_i\, u_i, \quad \mathbf{m}_2^\top \mathbf{P}_i^h = w_i\, v_i, \quad \mathbf{m}_3^\top \mathbf{P}_i^h = w_i$$

where homogeneous world point $\mathbf{P}_i^h = [X_i, Y_i, Z_i, 1]^\top$.

Q2.1 — [2 points]

Eliminate the unknown scale w_i. Write the resulting equations that are *linear* in the 12 m_{ij} entries of $\mathbf{M}$, and form a constraint by equating them to zero.

TODO: Your two linear equations here.

Q2.2 — [2 points]
Stacking your equations for N point correspondences gives a homogeneous system $\mathbf{Am} = \mathbf{0}$, where $\mathbf{m}$ is the 12-entry column vector of $\mathbf{M}$'s entries and $\mathbf{A}$ is a $2N \times 12$ matrix.

As $\mathbf{M}$ operates on homogeneous coordinates, any non-zero scalar multiple $c\mathbf{m}$ produces the exact same 2D pixel coordinates. Therefore, $\mathbf{m}$ is only defined 'up to scale'.

How many degrees of freedom does $\mathbf{M}$ have? What is the minimum number of equations needed to find a unique solution for $\mathbf{M}$ (up to scale)? Why is that? How many point correspondences N is that?

TODO: Your answer here.

SVD. To avoid the trivial solution $\mathbf{m} = \mathbf{0}$, we wish to enforce a unit scale constraint: $||\mathbf{m}|| = 1$. We solve this constrained minimization using the *Singular Value Decomposition* (SVD) of $\mathbf{A}$, written as $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. So, what is the SVD?

We can think of the SVD as a generalization of Principal Component Analysis (PCA). In PCA, we perform dimensionality reduction by finding a projection vector $\mathbf{w}$ *that maximizes the variance of our data* $\mathbf{X}$*, subject to* $||\mathbf{w}||^2 = 1$*. Then, by setting the derivative of the Lagrangian to zero, we found that* $\mathbf{w}$ *must be an* ***Eigenvector*** *of* $\mathbf{X}^\top\mathbf{X}$*. The optimal projection vector* $\mathbf{w}$ *corresponds to the one with the largest* ***Eigenvalue*** λ*.*

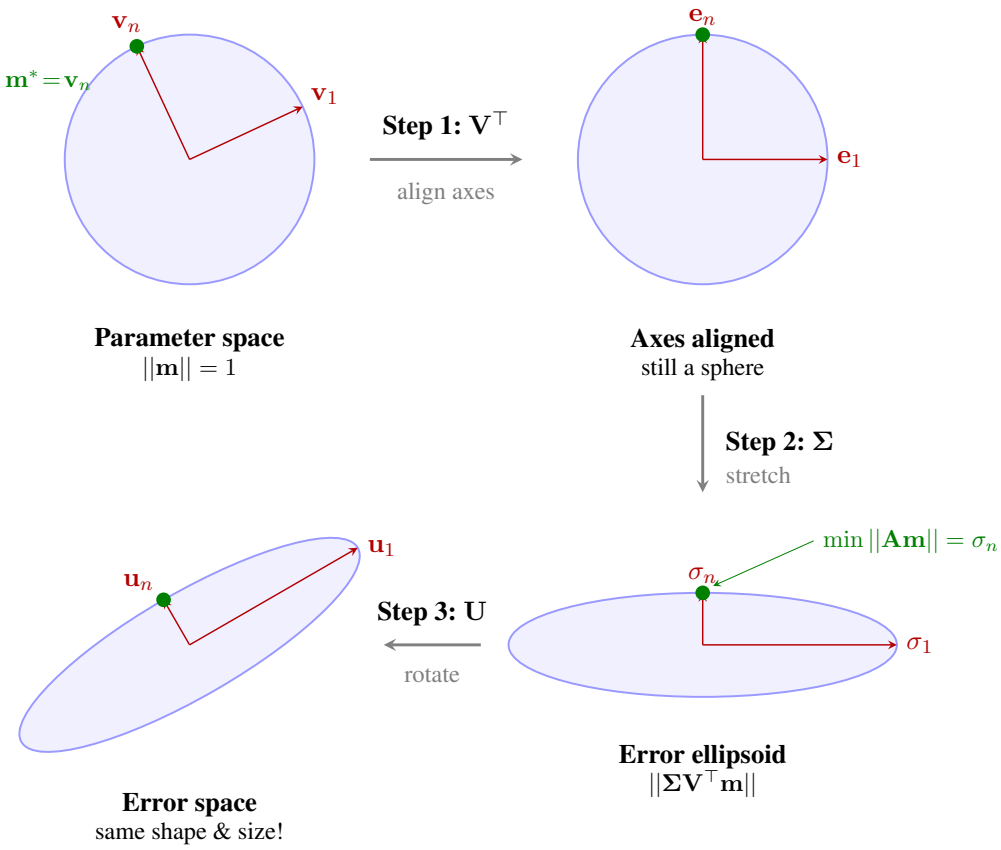
Our camera calibration shares this scale constraint: because $c\mathbf{m}$ produces the exact same image as $\mathbf{m}$, we restrict our search to the 12D unit sphere ($||\mathbf{m}|| = 1$) and seek to minimize the algebraic error $||\mathbf{A}\mathbf{m}||^2$. Because $\mathbf{A}\mathbf{m} = \mathbf{0}$ means our projection equations are perfectly satisfied, we want the unit vector $\mathbf{m}$ that makes this residual as small as possible. This is the opposite of PCA, where we sought the direction of maximum variance.

Recall that $\mathbf{A}$ is a $2N \times 12$ matrix that maps points from our 12D parameter space into a $2N$-dimensional error space. While Eigendecomposition only works when mapping a space to itself (square matrices), the SVD ($\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$) explicitly describes how $\mathbf{A}$ transforms our parameter unit sphere into an error ellipsoid in three distinct geometric steps (read right-to-left):

- **Step 1: $\mathbf{V}^\top$ (Alignment).** $\mathbf{V}$ is an orthogonal matrix whose columns ($\mathbf{v}_i$) are the eigenvectors of $\mathbf{A}^\top\mathbf{A}$ that form an orthonormal basis for our 12D parameter space. Multiplying by $\mathbf{V}^\top$ rotates our unit sphere so that its axes align with the principal axes of our system. As it is a rigid rotation, the sphere remains a sphere ($||\mathbf{V}^\top\mathbf{m}|| = 1$).
- **Step 2: $\mathbf{\Sigma}$ (Stretching).** $\mathbf{\Sigma}$ is a diagonal matrix of singular values ($\sigma_1 \geq \cdots \geq \sigma_{12} \geq 0$). Their squares ($\sigma_i^2$) are the eigenvalues of $\mathbf{A}^\top\mathbf{A}$. It scales the rotated sphere along each axis i by σ_i to become an ellipsoid. The singular values dictate the lengths of the radii of this error ellipsoid.
- **Step 3: $\mathbf{U}$ (Rotation).** $\mathbf{U}$ is an orthogonal matrix that takes this stretched ellipsoid and rigidly rotates it into our $2N$-D error space. Because $\mathbf{U}$ represents a rigid rotation, it does not change the size or shape of the ellipsoid, meaning it doesn't change the magnitude of the error vectors:

$$||\mathbf{A}\mathbf{m}||^2 = ||\mathbf{U}(\mathbf{\Sigma}\mathbf{V}^\top\mathbf{m})||^2 = ||\mathbf{\Sigma}\mathbf{V}^\top\mathbf{m}||^2$$

For example, in a 2D parameter space, the mapping to the error space would look like this. The green dot tracks the optimal $\mathbf{m}^* = \mathbf{v}_n$ throughout.



Q2.3 — [3 points]
To calibrate **Camera A**, we run `U, S, VT = np.linalg.svd(A)` on our $2N \times 12$ matrix $\mathbf{A}$ built from a photo of a 3D calibration stage with 20 point correspondences. The twelve singular values returned are:

$$S = [\ 142.5,\ 89.3,\ 71.0,\ 54.2,\ 41.8,\ 33.1,\ 19.7,\ 12.4,\ 5.6,\ 2.1,\ 0.83,\ 0.0003\ ]$$

Which SVD outputs give us our desired calibrated camera matrix $\mathbf{M}$?
Place ■ next to all correct answers.

(a) ☐ Any row of $\mathbf{V}^\top$. All are unit vectors ($||\mathbf{m}|| = 1$) that align the unit sphere with the principal axes, so all satisfy the scale constraint and give valid cameras.

(b) ☐ First row of $\mathbf{V}^\top$. It aligns the unit sphere with the principal axes, and corresponds to $\sigma_1 = 142.5$.

(c) ☐ Last row of $\mathbf{V}^\top$. It aligns the unit sphere with the principal axes, and corresponds to $\sigma_{12} = 0.0003$.

(d) ☐ The diagonal vector from S. The singular values are the camera parameters as they encode how it stretches the scene along each axis.

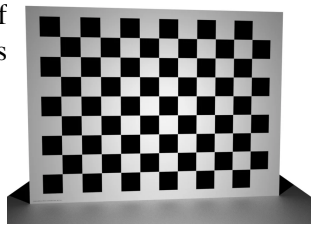
(e) ☐ Any column of $\mathbf{U}$. All are unit vectors ($||\mathbf{m}|| = 1$) that align the ellipsoid with the error space, so all satisfy the scale constraint and give valid cameras.

(f) ☐ First column of $\mathbf{U}$. It aligns the ellipsoid with the error space, and corresponds to $\sigma_1 = 142.5$.

(g) ☐ Last column of $\mathbf{U}$. It aligns the ellipsoid with the error space, and corresponds to $\sigma_{12} = 0.0003$.

Q2.4 — [3 points]

You calibrate a second camera (**Camera B**) using a photo of a 2D calibration checkerboard with 20 corners. Here are its singular values:



$$[\,310.4,\ 198.7,\ 155.2,\ 88.6,\ 61.3,\ 42.0,\ 27.5,\ \ldots$$
$$14.1,\ 0.0006,\ 0.0004,\ 0.0002,\ 0.0001\,]$$

Both cameras A and B have tiny algebraic residuals $||\mathbf{Am}||$, yet when tested on new scene points, the reprojection error in Camera B is 200 px.

(i) Something is wrong with Camera B's calibration. What is it, why does it cause the large reprojection error on the test point, and how can you tell from the singular values? **[3–5 sentences]**

TODO: Your answer here.

Q2.5 — [3 points]

A classmate suggests applying *Hartley normalization* to reduce the reprojection error in Camera B. Place ■ next to all correct answers.

(a) ☐ It will help. The issue is caused by the large magnitude mismatch between pixel coordinates and world coordinates in $\mathbf{A}$. Normalization will rescale the entries so that the SVD can resolve all 12 singular directions accurately.

(b) ☐ It will help, but only if you also add more point correspondences from the same checkerboard to overdetermine the system further.

(c) ☐ It won't help. Camera B's problem is a geometric degeneracy and no preprocessing on $\mathbf{A}$ can fix it.

(d) ☐ It won't help. Hartley normalization only applies to the 8-point algorithm for the fundamental matrix, not to the DLT for camera calibration.

SVD Summary. Camera geometry estimation problems via DLT use the SVD everywhere: camera matrices, essential and fundamental matrices, point triangulation. Why:

- **Enforces the scale constraint:** It lets us search only valid solutions on the unit sphere ($||\mathbf{m}|| = 1$) to homogeneous problems, preventing the useless trivial solution $\mathbf{m} = 0$.
- **Finds the optimal parameters:** The columns of $\mathbf{V}$ provide an orthogonal basis of candidate solutions for our camera matrix with known error residuals to pick from.
- **Quantifies the error:** The singular values (σ_i) tell us exactly what the algebraic error $||\mathbf{Am}||$ will be if we choose the corresponding column of $\mathbf{V}$.
- **Ensures numerical stability in a computer:** It operates on $\mathbf{A}$ directly, so avoids floating-point precision loss from computing the eigenvectors of $\mathbf{A}^\top\mathbf{A}$ directly.
- **Handles overdetermined, non-square systems:** Our matrix $\mathbf{A}$ is typically tall and skinny as we have many point correspondences and few parameters. To solve the homogeneous form $\mathbf{Am} = \mathbf{0}$, the SVD provides the approximate right null space (the last column of $\mathbf{V}$). If we instead formulate it as an inhomogeneous system $\mathbf{Ax} = \mathbf{b}$ (e.g., by setting $M_{3,4} = 1$), the SVD constructs the Moore-Penrose pseudoinverse $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ to find the least-squares solution $\mathbf{x} = \mathbf{A}^+\mathbf{b}$. This is exactly what `lstsq` computes under the hood!

Minor Python note: `np.linalg.svd` returns a matrix called `Vh`; short for $\mathbf{V}^H$. This is the *conjugate transpose* (or *Hermitian*) of $\mathbf{V}$. For real-valued matrices like ours, $\mathbf{V}^H = \mathbf{V}^\top$.

Q3 — [14 points]
In the code, you will consider depth from two camera views using two pipelines that rely on *homographies* in different ways:

- **Part A (Calibrated):** Given projection matrices $\mathbf{M}$, the *plane-sweep* algorithm warps Camera 1's image into Camera 2's view via a depth-dependent homography $\mathbf{H}(\lambda)$ to help us match.
- **Part B (Uncalibrated):** Given only feature point correspondences, we can estimate the fundamental matrix $\mathbf{F}$ and from that derive *rectifying homographies* $\mathbf{H}_1, \mathbf{H}_2$ to help us match.

Both pipelines recover scene geometry information but with different guarantees. What can each approach tell us about the scene?

This is challenging! *Draw these out visually* and use the *livePlaneSweep.py* demo to help your understanding.

Q3.1 — [2 points]

Warm-up: Which scenarios cause *both* the calibrated and the uncalibrated pipelines to fail to estimate scene geometry?
Place ■ next to all correct answers.

(a) ☐ Scenes where all objects are more than 100 m away. Both methods require nearby geometry.

(b) ☐ Scenes with textureless regions. Both methods rely on local photometric matching to establish correspondences, and a uniform surface provides no distinctive pattern.

(c) ☐ Scenes lit by a single point light source. Both methods require complex multi-directional illumination.

(d) ☐ Cameras with different focal lengths. Both methods require identical intrinsics (even if they are unknown).

(e) ☐ Cameras placed at the same position with different orientations. With zero baseline there is no parallax, so neither method can recover depth.

Part A: Calibrated. In lecture, we saw that we can write each camera as $\mathbf{M} = [\mathbf{A} \mid \mathbf{t}]$ with camera center $\mathbf{C} = -\mathbf{A}^{-1}\mathbf{t}$. A pixel in Camera 1 $\mathbf{p} = [u, v, 1]^\top$ back-projects to the 3D point $\mathbf{P} = \mathbf{C}_1 + \lambda\, \mathbf{A}_1^{-1}\mathbf{p}$ along ray $\mathbf{r} = \mathbf{A}_1^{-1}\mathbf{p}$, with $\mathbf{p}$ at depth λ. Projecting $\mathbf{P}$ into Camera 2 gives a depth-dependent homography:

$$\mathbf{H}(\lambda) = \lambda\, \mathbf{B} + \mathbf{a}\, \mathbf{e}_3^\top, \qquad \mathbf{B} = \mathbf{A}_2\, \mathbf{A}_1^{-1}, \qquad \mathbf{a} = \mathbf{A}_2\, \mathbf{C}_1 + \mathbf{t}_2$$

This geometric relationship is used by the fronto-parallel *plane-sweep algorithm* to evaluate $\mathbf{H}(\lambda)$ at many candidate depths. We warp Camera 1's *screen space point coordinates* into Camera 2's view. Then, in *pixel intensity space*, we measure local patch agreement by some metric like normalized cross-correlation (NCC). For each point, the best depth λ is kept.

Q3.2 — [2 points]

$\mathbf{H}(\lambda)$ is a single 3×3 matrix applied to Camera 1's coordinates. For which screen space points does this geometric warping produce the correct correspondence in Camera 2?
Place ■ next to all correct answers.

(a) ☐ All points, because $\mathbf{H}(\lambda)$ is derived from the full projection matrices of both cameras and so every point has correct depth.

(b) ☐ Points near the image center, where the pinhole model is most accurate.

(c) ☐ Points whose depth is close to λ. The homography is a local approximation that holds for points near that depth and degrades for farther points.

(d) ☐ Points whose depth is λ. The homography is geometrically exact only for points at that depth.

(e) ☐ Points with sufficient local texture in the pixel image, because $\mathbf{H}(\lambda)$ needs texture gradients to determine where each pixel maps.

Q3.3 — [4 points]

As the depth plane approaches Camera 1's center ($\lambda \to 0$), $\mathbf{H}(\lambda) \to \mathbf{a}\,\mathbf{e}_3^\top$. This means that every Camera 1 point $\mathbf{p}$ converges to the same location $\mathbf{a}$ in Camera 2. Where is $\mathbf{a}$ geometrically?
Place ■ next to all correct answers.

(a) ☐ All points converge to the **principal point** of the other camera, because at zero depth the image becomes infinitely small.

(b) ☐ All points converge to the **midpoint** between the two camera centers, since at zero depth the scene is "between" the cameras.

(c) ☐ All points converge to the **epipole** of Camera 1 in Camera 2, because at zero depth every back-projected ray collapses to Camera 1's center $\mathbf{C}_1$.

(d) ☐ All points converge to an **undefined** location because $\mathbf{H}(\lambda)$ is singular ($\det = 0$), so the limit has no geometric meaning.

Part B: Uncalibrated. Without known camera matrices $\mathbf{M}$, we estimate the 3×3 fundamental matrix $\mathbf{F}$ from 2D–2D point correspondences. It satisfies the *epipolar constraint*: for corresponding points $\mathbf{x}$ and $\mathbf{x}'$,

$$\mathbf{x}'^{\top} \mathbf{F}\, \mathbf{x} = 0$$

The product $\mathbf{l}' = \mathbf{F}\,\mathbf{x}$ is the *epipolar line* in image 2 on which the correspondence $\mathbf{x}'$ must lie.

Q3.4 — [2 points]

From $\mathbf{F}$, we can compute *rectifying homographies* $\mathbf{H}_1$ and $\mathbf{H}_2$ that help with dense matching. What do these homographies accomplish?
Place ■ next to all correct answers.

(a) ☐ They warp both images so that each point's correspondence appears at the same image coordinates, eliminating the need for further matching.

(b) ☐ They decompose $\mathbf{F}$ into the cameras' relative rotation and translation, so that correspondences can be triangulated directly.

(c) ☐ They warp both images to remove radial lens distortion, since $\mathbf{F}$ was estimated from distorted measurements, so that we can match along 1D epipolar lines.

(d) ☐ They warp both images so that each point's correspondence lies on the same horizontal row, so that we can match along 1D epipolar lines.

Q3.5 — [4 points]

After rectification, the disparity d at a pixel is the offset to its correspondence. For calibrated cameras with baseline b and focal length f, depth satisfies $Z = bf/d$. In Part B (uncalibrated), you estimate $\mathbf{F}$ from 2D correspondences alone and then rectify. Which statements about the recovered disparity are correct?
Place ■ next to all correct answers.

(a) ☐ Disparity is proportional to depth: closer objects have smaller disparity.

(b) ☐ Disparity is inversely proportional to depth: closer objects have larger disparity.

(c) ☐ Disparity from uncalibrated rectified images has no systematic relationship to depth, because the rectifying homographies introduce arbitrary distortion.

(d) ☐ Disparity after rectification can give metric depth, because the rectifying homographies standardize the geometry.

(e) ☐ Disparity after rectification can give relative depth, because b and f are unknown.

Feedback? (Optional)

We appreciate your feedback on how to improve the course. You can provide anonymous feedback through this form which can be accessed using your Brown account (your identity will not be collected). If you have urgent non-anonymous comments/questions, please email the instructor.